

DirZero C++ Rewrite - Current Work Targets by Team Member

1. Team Overview

This document is the actual technical work list. It is meant to tell each person what they need to build in code, not just the overview like thingyy.

The app is being rebuilt in C++ with Qt6 and remote SSH/SFTP access. The goal is to make the system modular and testable while keeping the Python app as the reference behavior.

2. Shubhneek - Core Architecture, Permission Design, and Final Integration

Actual technical work to build

I am responsible for assembling architecture pieces.

Create and own the following:

1. Project foundation

- `CMakeLists.txt`
- project root structure
- Qt6 dependency wiring
- build setup for Linux and future portability

2. Shared application state models

- `MachineInfo.hpp/.cpp`
- `MachineState.hpp`
- base enums and status definitions for online, offline, auth required, connected, etc.

3. Permission policy layer

- `PermissionPolicy.hpp/.cpp`
- `AccessRule.hpp`
- `AuthState.hpp`
- logic that decides whether a file operation is allowed before it happens

4. App integration layer

- `Dir2ZeroWindow.hpp/.cpp`
- final wiring of all modules into one working window
- state flow between discovery, auth, file model, and UI

5. Build stability and review

- fix compile errors and broken interfaces

- ensure all modules agree on shared data structures
- validate the final application logic before merge

Objective

This work should produce a stable architecture where all the other team members plug into shared interfaces instead of creating inconsistent models.

3. Dhvani - Credential Security and File Operation Permission Layer

Actual technical work to build

Dhwani owns the security side of the project.

Create and own the following:

1. Credential persistence

- `CredentialStore.hpp/.cpp`
- store username, password, and SSH private key path locally
- protect config files with restricted permissions
- read and return credentials cleanly

2. Permission enforcement logic

- `PermissionPolicy.hpp/.cpp`
- `AccessRule.hpp`
- define allowed/disallowed file actions
- block destructive or unauthorized operations before execution

3. Auth and access state handling

- `AuthState.hpp`
- values for successful auth, missing creds, invalid key, permission denied, etc.

4. Operation gating helpers

- a function/class that validates:
 - create file
 - create directory
 - rename
 - delete
 - copy
 - move
 - paste
- return a clear permission failure reason when blocked

Objective

Dhwani's module must make the app safe. The backend should not perform destructive operations without permission validation.

4. Ritesh - Remote Discovery, Machine State, Filesystem Model, and Connectivity Engine

Actual technical work to build

Ritesh owns the remote machine and filesystem access layer.

Create and own the following:

1. Tailscale peer discovery

- `TailscaleDiscovery.hpp/.cpp`
- parse the `tailscale` command output or JSON response
- detect all reachable peers
- map peer names, IPs, and metadata

2. Machine information and health model

- `MachineInfo.hpp`
- `MachineState.hpp`
- keep track of:
 - online/offline
 - SSH reachable/unreachable
 - auth required
 - connected
 - error state

3. Reachability probe layer

- `Probeworker.hpp/.cpp`
- check whether port 22 is reachable over TCP
- detect machines that are online but not SSH-enabled

4. State refresh flow

- a function/class that refreshes the machine list periodically
- updates status without freezing the UI thread

5. Filesystem tree model

- `FSNode.hpp/.cpp`
- represent remote folders and files in a tree structure
- support nested directories and lazy loading placeholders
- keep parent/child relationships and paths consistent

6. SFTP connection and listing layer

- `SFTPManager.hpp/.cpp`

- connect to SSH/SFTP sessions
- list directories and read filesystem metadata
- handle connection failures and empty directory cases

Objective

This module should answer: "Which machines are alive, which are reachable, and how do we read and model the remote filesystem safely?"

5. Eka - UI Shell, Panels, Tree View, and User Action Flow

Actual technical work to build

Eka owns the visible app and user interaction flow.

Create and own the following:

1. Main window shell

- `Dir2ZeroWindow.hpp/.cpp`
- top-level window setup
- toolbar/status area
- layout host for machine panels

2. Machine panel widget

- `MachinePanelWidget.hpp/.cpp`
- one card/panel for one remote machine
- connect/disconnect controls
- status display
- auth-related actions

3. File tree view widget

- `RemoteFileTreeView.hpp/.cpp`
- tree UI for browsing remote directories
- expand/collapse behavior
- actions for open, rename, delete, refresh

4. Auth UI flow

- `MachineAuthBar.hpp/.cpp`
- username, password, key path entry
- connect interaction
- auth error display

5. Theme manager and styling

- `ThemeManager.hpp/.cpp`
- load `.qss` themes
- apply current theme to the app

- support switching between themes at runtime

Objective

This module should answer: “How does the user see and interact with the machines and files?”

6. Jaideep - Shared Utilities, Metadata Plumbing, and Integration Support

Actual technical work to build

Jaideep is not taking the main UI ownership. He supports the app by keeping shared data and integration logic clean.

Create and own the following:

1. Helper metadata utilities

- `RemoteClipboard.hpp/.cpp`
- clipboard and drag/drop helper logic for file operations
- data transfer payload format for app actions

2. Layout/helper widgets

- `ResponsiveGridContainer.hpp/.cpp`
- resize-friendly machine panel layout helper

3. State formatting support

- helpers for converting file metadata into display strings
- size formatting
- date formatting
- permission display formatting

4. Integration validation support

- help verify the UI and backend agree on states
- ensure metadata passed between modules is consistent
- catch mismatches in state names, paths, and identifiers

Objective

This module keeps the app from breaking at the boundaries between UI and backend.

7. Suggested Module Set for the Entire Team

A practical target structure:

```
DirZero/  
├─ CMakeLists.txt
```

```
src/
├── main.cpp
├── core/
│   ├── MachineInfo.hpp
│   ├── MachineInfo.cpp
│   ├── MachineState.hpp
│   ├── PermissionPolicy.hpp
│   ├── PermissionPolicy.cpp
│   ├── AuthState.hpp
│   ├── AccessRule.hpp
│   └── AccessRule.cpp
├── security/
│   ├── CredentialStore.hpp
│   └── CredentialStore.cpp
├── discovery/
│   ├── TailscaleDiscovery.hpp
│   ├── TailscaleDiscovery.cpp
│   ├── Probeworker.hpp
│   └── Probeworker.cpp
├── fs/
│   ├── FSNode.hpp
│   ├── FSNode.cpp
│   ├── RemoteFSModel.hpp
│   ├── RemoteFSModel.cpp
│   ├── SFTPManager.hpp
│   └── SFTPManager.cpp
├── ui/
│   ├── MachineAuthBar.hpp
│   ├── MachineAuthBar.cpp
│   ├── MachinePanelWidget.hpp
│   ├── MachinePanelWidget.cpp
│   ├── RemoteFileTreeView.hpp
│   ├── RemoteFileTreeView.cpp
│   ├── Dir2ZeroWindow.hpp
│   ├── Dir2ZeroWindow.cpp
│   ├── ThemeManager.hpp
│   └── ThemeManager.cpp
├── helpers/
│   ├── RemoteClipboard.hpp
│   ├── RemoteClipboard.cpp
│   ├── ResponsiveGridContainer.hpp
│   └── ResponsiveGridContainer.cpp
└── app/
    └── AppController.hpp
```

This is only a suggested layout. The real rule is: each teammate should own concrete files and not work on unrelated areas by guessing.

8. Deliverable Rule

Every teammate should produce one working deliverable from their assigned module set.

Minimum expected result:

- working implementation draft
 - clear API or class boundaries
 - proof that the feature works in a focused test or compile check
 - short summary of what was built and where it connects to the app
-

9. Final Goal

The final app should be built from these independent pieces:

- discovery engine
- connectivity engine
- security and permission layer
- remote filesystem manager
- UI shell and user actions
- shared support utilities

Once all these are connected, we could assemble/compile all these into a final shippable product.